

Periscope: how it is built and why

Super parsers for your competitors. Every page, every click, every country, then past the login. Built on Steel cloud browsers at the UTMIST x WAT.ai Battle of the Schools, September 12-13, 2026.

The idea in one paragraph

Every research tool reads what a website serves. A fetch, a search index or a reader API gets the HTML the server sends. The facts that matter on a modern SaaS site are behind client-side state: the annual toggle, the compare accordion, the seat dropdown, a tooltip, a show-more button, an iframe, a JSON call on page load, a cookie wall that differs by country, and the login. Periscope drives real browsers on Steel to perform those actions, sees the page from several countries and devices at once, keeps the session alive when a wall appears so a person can clear it in Steel's live view, and signs in with credentials that live in Steel's vault rather than in the model. Every line it records carries the action that revealed it, the country and device it was seen from, and the Steel session that produced it.

The stack, and why each piece

Piece	What it does in Periscope	Why we chose it
Steel	Cloud Chrome over CDP, proxies by country, device emulation, live player, CAPTCHA solver, credentials vault, profiles, extensions, files and traces	The only part we could not replace with a library. Every demo beat uses a Steel feature a plain browser does not have.
TypeScript on Node 24	Everything that touches browsers, storage and the API; pnpm workspaces with a shared types package	Steel, Playwright and Stagehand are TypeScript-first. One language, one contract for three people building in parallel.
playwright-core	The deterministic reveal strategies and the walker	Fast, cheap, repeatable on stage. Needs no model and produces every number in the demo.
Stagehand v4	Model-driven actions on top, opt-in	Attaches to Steel through the uploaded extension; not yet stable with the rest of the pipeline, so the demo does not depend on it.
Claude Opus 5	Feature matrix and diff summary through forced tool schemas	Rules cannot name features or summarise change. Every extracted row must cite observation ids or it is rejected.
SQLite + Qdrant	System of record with versioned migrations and micro-dollar accounting; vectors for search	One file to hand a judge, transactional writes, exact money; vectors only where search needs them.
node:http API	JSON routes, server-sent events, idempotency keys, live sessions	No framework to configure on stage; the front end and the fixtures speak the same contract.
Streamlit (Python)	The front end: live browsers, story cards, Steel usage trace, intelligence, benchmark	Screens in hours; it talks only to the API, so the same screens work on fixture responses.
Vitest	79 offline tests: fixtures in a local Chrome, in-memory SQLite, fake Steel, fake model	Runs anywhere without keys; live tests are a separate config.
Next.js on Vercel	Helix Ledger, our own test SaaS with 65 planted facts, a cookie wall by country and an anti-bot login	A controlled target makes the benchmark fair and repeatable.

How one run flows through the eight layers

- **1. Request.** The front end posts a run: competitor, url, pages, job types, countries, optional account. The API records it and launches it in-process, so several runs share one Steel pool.
- **2. Coordinator.** Jobs are ordered walkers first, then reveal, borders, surface and benchmark. Every job writes events through one sink.
- **3. Browser layer.** The pool leases Steel sessions: ten at a time, one per account, an 11-minute autonomous deadline with a checkpoint, forced release at 14 minutes, a crash journal so a restart releases what a dead process left open. Each session is created with the right proxy country, device, profile, credential namespace and, for model runs, Stagehand's extension.
- **4. Surface and benchmark.** Steel's scrape shows the page as a fetch tool sees it; a plain HTTP fetch is stored as its own sightings. That is the left side of every comparison, and it says 'uncertain' when it failed.
- **5. Reveal.** A fixed strategy set runs in the browser: decline the cookie wall, click tabs and accordions, select every option, flip switches, show more and scroll, hover tooltips, open modals, read iframes, collect documents, record page-load JSON calls. After each action the visible text is diffed line by line; new lines become hidden observations, flagged 'missed by fetch' when the surface never had them. A URL guard restores the page after any click that navigated.
- **6. Borders.** The same page opened in parallel from Canada, the United States and Germany, desktop and mobile, through Steel proxies. A grid shows what each country saw that the others did not and whether prices differ.
- **7. Walker and login.** A crawl of same-origin links, one observation per visible line per screen, a wall check after every navigation. On a login wall with an account, Steel has already injected the vaulted credentials; the walker verifies the anti-bot box in the browser, presses the form's own submit, and continues on the dashboard.
- **8. Human in the loop.** Walls are classified from the DOM. On a CAPTCHA, Steel's solver runs first for 45 seconds; only then the handoff fires: the session stays alive, the live view link is posted, a person clears the wall in the same browser, a generation-checked resume continues the walk. A ten-minute timer abandons it.
- **9 to 12. Knowledge, intelligence, API, front end.** Every event lands in SQLite transactionally. Pure functions compute coverage, the borders grid, prices, the run-to-run diff; Claude fills the feature matrix with cited rows. The API exposes it all plus the live sessions. The front end puts the browsers at the centre, with a trace that names every Steel feature as it is used.

The decisions and why we made them

Decision	Why
Deterministic first, model second	Rules are fast, cheap and repeatable on stage. The model adds judgement only where rules cannot. The demo ran all night on Steel alone with no Claude key and still produced every number.
Observation ids scoped to run and source	The first id hashed only competitor, url, vantage and text, so a second run recorded nothing and a diff had nothing to compare. Adding run and source fixed the diff and lets the fetch benchmark and the browser both keep their sighting of the same line.
Honesty over inflation	The counter counts distinct lines per page, never per vantage. The benchmark says 'uncertain' when the fetch failed. The matrix never marks a feature 'working', only 'observed'. Text found inside a page is data, never an instruction.
A wall must ask for something	KYC vendors describe identity verification on every marketing page, so form-style walls need a visible input. A sign-in form with a password field is a login wall even with a CAPTCHA widget inside. A CAPTCHA wall needs widget markup or visible challenge text; otherwise Steel's own solver scripts made every page look like a CAPTCHA.
Steel first, human second	The solver gets 45 seconds before anyone is called. The human acts inside the same Steel session, so the walk resumes from the same page instead of starting over.
Never a password in our code	Credentials go into Steel's vault through a hidden prompt; Steel types them into the form. Periscope never creates accounts, never enters card details, never presses pay, buy, delete, invite, upgrade, checkout or log out. Billing pages are read, never operated.
Three owners, one frozen contract	The shared types package was frozen early so three people could build in parallel and merge in a fixed order. Every module has one owner; touching another owner's file is a reviewed pull request.
A controlled target	Helix Ledger exists so the benchmark is fair: 65 planted facts, three country variants, an anti-bot login. Periscope finds 63; a fetch finds 22; a reader connector finds 21; a browsing agent found 38. The gap is exactly the content behind actions.
Trade-offs we accepted	Steel profiles did not persist cookies in our tests, so login relies on injection every time. The CAPTCHA solver is not deterministic. Stagehand stays opt-in. The plan allows ten sessions, so launching every competitor at once queues. A vaulted credential is bound to its exact origin.

What the judges see, in order

- **Parse.** One Steel browser on the Helix pricing page: 23 lines a fetch tool never returned, in about 12 seconds, each labelled with the action that revealed it.
- **Three countries.** Six Steel browsers through CA, US and DE proxies on desktop and mobile. Canada sees CA\$ prices and a GST line, Germany sees euro prices behind a cookie banner the agent declines, the US sees dollars.
- **Log in.** Steel injects the vaulted credentials, the walker verifies the anti-bot box, signs in, and reads the dashboard: seats used, bank connections, integration badges, data region, SSO note, card ending. No human involved; if any step fails the card says where and a person takes over in the embedded live view.
- **Below the browsers.** The Steel usage trace, one story card per run, coverage, countries, prices, the feature matrix, and the benchmark against fetch, a reader connector and a browsing agent.

Numbers to remember

Measure	Value
Hidden lines on the Helix pricing page a fetch never returned	23 (18 planted, plus a document, an API url and extra option lines)
Planted facts found on Helix, 65 total	Periscope 63; browsing agent 38; Claude Opus with fetch 22; Claude Opus with reader connector 21
Dashboard facts behind the login, autonomous sign-in	7 of 8, no human, about a minute including the crawl
Borders on Spotify premium, six browsers	CA \$13.99, US \$12.99 with Hulu, DE 12,99 EUR, in 24 seconds
Offline tests / live tests	79 passed / session, proxies, profiles, credentials, traces, files, Turnstile solver verified live

Repository: github.com/MightyTMZ/periscope. Contract: docs/api.md. Demo commands: docs/demo-runbook.md. Specification with test ids: docs/periscope-final-architecture.md.

Questions Steel's engineers will ask, and the answers

Short answers we can say on stage. Longer detail is in the code and the runbook.

Question	Answer
Why Steel instead of local Playwright or another cloud browser?	Five things we use that a local browser does not have: proxies by country, a live player a person can act in, a credentials vault that types the password for us, a CAPTCHA solver we try first, and session profiles. We also load Stagehand's extension through Steel's extensions API.
How do you stay inside the ten-session limit?	A pool with ten slots and a queue, one session per account, an 11-minute autonomous deadline with a checkpoint, a forced release at 14 minutes. Borders uses a light strategy set so six browsers finish in under a minute.
What happens when your process crashes with sessions open?	Every lease is journaled to disk. On the next start the pool lists Steel's live sessions, releases the ones it owned, and the run is marked partial. Walkers checkpoint every five screens.
Why rules instead of an LLM agent driving the browser?	Rules are deterministic, cost nothing per action and are fast enough to run live. They find all 18 planted lines. The model is used where rules cannot judge: naming features and summarising change. Stagehand is wired and opt-in.
How do you stop the agent from doing damage on a real site?	A blocklist on labels and urls (pay, buy, delete, remove, send, invite, publish, upgrade, subscribe, submit, checkout, update payment, log out), a URL guard that restores the page after any navigating click, billing pages read-only, and no form submission except the sign-in with vaulted credentials.
Where is the password?	In Steel's credentials vault, bound to the exact origin and a namespace per account. Steel injects it into the form; our code only waits for the field to be filled. Neither the model nor the repository ever holds it.
Why not persist the login with a Steel profile?	We tried four variants. Profiles reach READY, localStorage sometimes survives, cookies did not. So every walker session injects the credential again and keeps its profile as a bonus.
What if the CAPTCHA solver fails?	It gets 45 seconds after full page load. Then the handoff: the session stays alive, the live view link is posted, a person clears it in the same browser, and a generation-checked resume continues the walk. Self-hosted widgets like ALTCHA are cleared by the walker itself.
How did you verify the proxies are real?	Germany returned German text, euro prices and a German cookie banner; Canada returned CA\$ prices and a GST line. One surprise: Cloudflare challenged proxied traffic to our tunnel, so the login run opens without a proxy.
Isn't 'missed by fetch' easy to inflate?	It is a line-level diff against the surface text, counted once per page (max across vantages, never a sum), with navigation and code-like lines filtered, and the fetch stored as its own source. If the fetch failed the counter reads 'uncertain'.
How do you diff two runs of the same site?	Observation ids include the run, so both runs record every line. The diff keys on url, vantage and text and only compares pages scanned in both runs, so a skipped page never shows as a removal.
Can every claim be traced?	Every observation carries the revealing action, the vantage, the Steel session id, the viewer url and a screenshot path. Matrix rows are rejected unless they cite observation ids that exist.
What does a run cost?	Steel session minutes, and Claude tokens metered as receipts in micro-dollars with per-run and per-job caps. The whole demo set ran under three dollars of Steel.
Does Stagehand actually work on Steel?	It attaches: we upload its runtime extension once through Steel's extensions API and connect by extension id; extract and act work in isolation. Inside the full reveal it still loses the page, so it is opt-in and not in the demo path.
Why is the fetch benchmark fair?	It is a plain HTTP fetch of the same url, stored separately. On hosts this machine cannot resolve it falls back to a hosted reader fetch, still a fetch. The rubric of 65 facts is fixed and published.
What breaks on a real site you have not seen?	Custom widgets without roles, unusual consent walls, and infinite pages past the scroll cap. The strategy set is a list that grows per incident; every miss becomes a fixture test.
Why SQLite and Qdrant, not one database?	SQLite is the system of record: transactional, one file, versioned migrations, exact money. Qdrant only serves search over text, filled from an outbox so it can be rebuilt from SQLite at any time.
How does the front end show live browsers?	Steel's session player url embeds in an iframe with no auth. The API lists live sessions with their run, purpose, country, device and current page; the page polls it every two seconds.
Is this allowed on real competitors?	We never create accounts, never enter card details, respect walls by handing them to a person, and only crawl what a logged-in customer sees. The demo target is our own site.
What would you build next with Steel?	Cookie-persisting profiles so a login is typed once, a solver for self-hosted anti-bot widgets, agent traces exported per finding, and a pool that spans regions so borders runs are not queued.